

LEZIONE 25/08/2026

1- NODE

Node.js è un runtime system open source multiplatforma orientato agli eventi per l'esecuzione di codice JavaScript, costruito sul motore JavaScript V8 di Google Chrome. Viene usato per realizzare applicazioni Web in JavaScript, permettendoci di utilizzare questo linguaggio, tipicamente utilizzato nella "client-side", anche per la scrittura di applicazioni "server-side".

Accenno Storico

Nato nel 1995 ad opera di Brendan Eich, JavaScript è stato per oltre un decennio confinato all'interno dei browser per rendere dinamiche le pagine web. La svolta è arrivata nel 2009, quando Ryan Dahl ha preso il potente motore V8 di Chrome e lo ha estratto dal browser, creando Node.js e permettendo a JS di girare direttamente sui server.

Analisi Concettuale della Definizione

Ecco la scomposizione logica e dettagliata delle parole chiave della definizione:

- **Runtime System (Ambiente di Esecuzione):** JavaScript nativamente sa fare ben poco da solo: ha bisogno di un "ambiente" che gli fornisca gli strumenti per interagire con il mondo esterno. Il browser è un runtime (offre strumenti come il DOM o l'oggetto `window`). Node.js è un altro runtime che gli fornisce strumenti diversi (come l'accesso al File System o alla rete del server o l'oggetto `global`).
- **Open Source:** Significa che il codice sorgente di Node.js è pubblico, gratuito e mantenuto da una vasta community globale. Chiunque può analizzarlo, contribuire o proporre migliorie.
- **Multiplatforma:** Il codice Node.js che scrivi sul tuo computer (sia esso macOS, Windows o Linux) eseguirà nello stesso identico modo su qualsiasi altro sistema operativo o server su cui è installato Node.js.
- **Orientato agli Eventi (Event-Driven):** Node.js funziona tramite un ciclo continuo chiamato *Event Loop*. Anziché bloccare l'esecuzione del programma mentre aspetta una risposta (es. la lettura di un file dal disco), registra un evento e continua a fare altro. Quando la risposta è pronta, scatena una funzione di callback.
- **Motore JavaScript V8:** È l'incredibile motore sviluppato da Google in C++ per Chrome. Prende il codice JavaScript (leggibile dagli umani) e lo compila direttamente in codice macchina (0 e 1, eseguibili dal processore) ad altissima velocità.
Nota: il motore **V8 non è semplicemente un compilatore**, ma un **sistema completo di esecuzione (engine)** che contiene al suo interno sia un interprete che un sistema di compilazione multilivello (ved. Approfondimento: V8 sotto).
- **Client-side vs Server-side:**
 - *Client-side:* Il codice gira sul dispositivo dell'utente (nel browser del PC o dello smartphone).
 - *Server-side:* Il codice gira sulla macchina remota (il server) che ospita il sito web, gestisce i database, autentica gli utenti ed elabora la logica aziendale

(Business Logic, ved. Approfondimento: Business Logic sotto) prima di inviare i dati al client.

Comando base: node <nome file js> - esegue il codice contenuto nel file js specificato all'interno del terminale.

2 - NPM e NODE

Quando installiamo node, viene installato anche un piccolo software di nome "npm" (Node Package Manager).

Come suggerisce la parola, è un "gestore di pacchetti".

Questo software ci libera dalla responsabilità di importare librerie e plugin vari manualmente (finora, ad esempio, abbiamo usato le CDN) e li gestisce per noi.

NPM genera un file package.json: un file chiave utilizzato in un progetto Node.js per descrivere le informazioni sul progetto stesso, le sue dipendenze, i comandi di script personalizzati e altre configurazioni. È un elemento fondamentale per la gestione dei pacchetti e la configurazione dei progetti [Node.js](#).

COMANDO	DESCRIZIONE
npm init	inizializzazione di NPM - Ci verranno fatte tutta una serie di domande che genereranno il file di configurazione base, package.json.
npm install <nome-pacchetto> (o npm i)	installazione dei pacchetti / librerie di terzi necessarie per il progetto
npm run start (nota per lo script "start" non è necessario scrivere "run", per tutti gli altri sì).	esegue il codice riportato dopo start nella sezione scripts del file package.json - es. "start": "node index.js " eseguirà il file index.js con node (nel terminale)
npm run watch	esegue il codice riportato dopo watch nella sezione scripts - es. "watch": "node -watch index.js " eseguirà il file index.js con node (nel terminale), <u>monitorando le modifiche in tempo reale (parola chiave watch dopo node e con i due trattini)</u>

Importante: una volta installato un pacchetto / libreria dobbiamo importarlo per poterlo utilizzare. Abbiamo a disposizione due sintassi:

CommonJS

```
const axios = require('axios');
```

ES6 Modules

```
import axios from 'axios'
```

Appendix- APPROFONDIMENTI

a- V8

Il percorso che il codice JavaScript compie all'interno di V8:

1. Parser: V8 legge il testo del codice JavaScript e lo trasforma in un albero concettuale chiamato *AST (Abstract Syntax Tree)*.
2. Interpretare (chiamato *Ignition*): V8 prende l'AST e lo converte rapidamente in *bytecode* (un codice intermedio) per iniziare a eseguirlo immediatamente senza attese.
3. Compilatore JIT (chiamato *TurboFan*): Mentre l'interprete esegue il codice, V8 monitora quali parti vengono eseguite più spesso (le cosiddette "hot functions"). Il compilatore prende queste parti, le ottimizza e le compila direttamente in codice macchina (0 e 1 per la CPU).

JavaScript si definisce un linguaggio JIT (Just-In-Time compiled): non viene compilato *prima* dell'esecuzione (come C o Rust), ma viene interpretato e compilato *durante* l'esecuzione stessa.

Vantaggio chiave: la riduzione dei tempi morti all'avvio. L'interprete inizia a eseguire il codice **all'istante**, senza dover attendere che l'intero programma sia compilato prima di partire. Per una pagina web o una serverless function in Node.js, l'avvio immediato è fondamentale.

Svantaggi principali:

1. Consumo di memoria (RAM): Tenere in memoria sia l'interprete che il compilatore JIT insieme al codice macchina ottimizzato richiede più risorse.
2. Prestazioni iniziali: Nelle primissime fasi di esecuzione, il codice interpretato è più lento rispetto al codice pre-compilato di C++. Solo dopo che la funzione diventa "hot", il compilatore JIT la velocizza portandola a prestazioni quasi pari a C++.

b- Business Logic

Nel gergo dello sviluppo software, il termine **"logica aziendale"** (in inglese *Business Logic*, detta anche *Domain Logic*) indica **l'insieme di regole, calcoli e decisioni che definiscono come l'applicazione deve comportarsi per risolvere il problema per cui è stata creata**, indipendentemente dall'interfaccia grafica e dal database.

Analisi Concettuale: Cos'è la Business Logic?

Per capire il concetto, separa un'applicazione in tre livelli principali:

1. **Interfaccia Utente (Presentation Layer):** Ciò che l'utente vede e clicca (pulsanti, form HTML, stile CSS).
2. **Database (Data Layer):** Dove risiedono i dati grezzi (tabelle, utenti, prodotti).
3. **Logica Aziendale (Business Logic Layer):** Il "cervello" dell'applicazione. Riceve gli input, applica le regole e decide cosa fare.

Il termine "aziendale" o "business" è un'eredità storica dell'ingegneria del software, nata quando la maggior parte dei sistemi informatici veniva sviluppata per digitalizzare processi aziendali (contabilità, gestione magazzino). Oggi, nel mondo dello sviluppo web, **"Business Logic"** significa semplicemente **"Logica di Dominio"** o **"Regole del Gioco"**.

LEZIONE 27/08/2026

BACKEND & EXPRESS JS

Backend, due strati:

1. **Server**
2. **DB**

Differenze principali tra frontend e backend: Front end esiste uno per utente - backend è uno solo, centralizzato!

Termini / Parole chiave che ci interessano:

- Payload: contenuto dati che ci viene trasmesso, che viene richiesto, o che noi inviamo
- Tipi di payload: diversi tipi, es HTML / text, ecc.
- Entry point: nel nostro caso l'entry point per implementazione logica saranno delle call back -> Programmazione orientata ad eventi.
- CRUD: Creare, rimpiazzare, aggiornare, eliminare dati.

Autenticazione / gestione delle API ed integrazioni (es. Stripe per pagamenti in un e-commerce): potrebbe esserci un backend che fa da intermediario globale con i vari server che utilizziamo.

Postman - funge da nostro client. Importante comprendere che siccome ci occupiamo backend non dobbiamo pensare alla pagina web ma al passaggio dati / alla trasmissione dei dati. Server / Web API sono i server che ci inviano la risposta a richiesta GET.

Express è una libreria che viene aggiunta ad un progetto node. Node per poter essere un runtime sensato deve potersi legare alle porte di rete del pc e stare in ascolto. Per costruire un server / backend / API, Express semplifica tante operazioni (di per sé si potrebbero creare anche senza ma diventerebbe non semplice al crescere della complessità

dell'applicazione) -> semplifica in particolare la gestione di dettagli tecnici che hanno proprio a che vedere con l'ascolto delle porte di rete.

EXPRESS

Express JS o semplicemente Express = framework che semplifica le operazioni più comuni nella creazione, e avvio di un server con node.

Nel dettaglio si tratta di un framework backend minimalista e veloce che offre funzionalità e strumenti solidi per lo sviluppo di applicazioni di backend scalabili.

Offre un sistema di routing e funzioni semplificate per estendere il framework sviluppando componenti e parti più potenti in base ai casi d'uso dell'applicazione.

Storia di Express

Express.js è stato fondato da TJ Holowaychuk. La prima versione, secondo il repository di Express.js su GitHub, risale al 22 maggio 2010, versione 0.12.

Nel giugno 2014, i diritti per gestire il progetto sono stati acquisiti da StrongLoop. Questa è stata poi acquisita da IBM nel settembre 2015.

Nel gennaio 2016, IBM ha annunciato che avrebbe posto Express.js sotto la supervisione dell'incubatore Node.js Foundation.

Uso di express:

il modo più semplice è installarlo tramite npm come qualsiasi pacchetto. Esiste anche un comando create, ma è meno pratico.

Quindi una volta fatto npm init e creato il nostro progetto (per convenzione [app.js](#)), lanciamo il comando npm install express.

Sarà poi necessario importarlo (con sintassi commonJS come sotto o attraverso la sintassi import in caso di ES modules):

```
const express = require('express'); // import express
const app = express() // inizializzazione di express invocandolo come una funzione e salvando il risultato in una variabile
const port = 3000 // port 3000 per convenzione
```

Nota: la variabile app conterrà un oggetto di Express e avrà quindi al suo interno tutta una serie di metodi che possiamo invocare per eseguire varie operazioni tipiche dei server, come per esempio:

- *Rotte HTTP*
- *Configurazione middleware*
- *Rendering view in HTML*
- *Registrazione di un template engine*
- *... molto altro*

```
app.get('/', (req, res) => { // definiamo la prima rotta
  res.send("Hello World!")
})
```

```
app.listen(port, () => { // avviamo il server, mettendolo in ascolto sulla porta indicata
```

```
console.log(`Example app listening on port ${port}`)  
})
```

Infine, possiamo creare i classici script per avviare il nostro server [app.js](#) con node (start e watch).

Porte

65k porte per ogni macchina/computer - servono come canali di comunicazione per l'esterno. Alcune porte sono definite dal protocollo. es. Https è la 443. Possono essere viste come delle frequenze radio.

Rotte

In Express JS, le rotte sono come indirizzi per indicare al server cosa fare quando riceve una richiesta su un certo percorso (URL).

Immagina un sito come una casa con diverse stanze: le rotte sono come le porte che portano a quelle stanze.

Ogni rotta è associata a un'azione specifica che il server deve compiere, ad esempio mostrare una pagina o inviare dei dati.

Nota: rotta "/" è la homepage del sito

Tramite Express possiamo definire tutti gli indirizzi (detti rotte) di cui abbiamo bisogno per la nostra applicazione. Tramite l'oggetto app, per ogni rotta possiamo specificare 3 elementi:

- metodo HTTP che dovrà essere usato per accedere alla rotta (per il momento get)
- path ossia l'URL della rotta
- funzione che ha lo scopo di rispondere quando viene richiamata la rotta

```
app.get('/', (req, res) => {  
  res.send('Hello World!')  
})
```

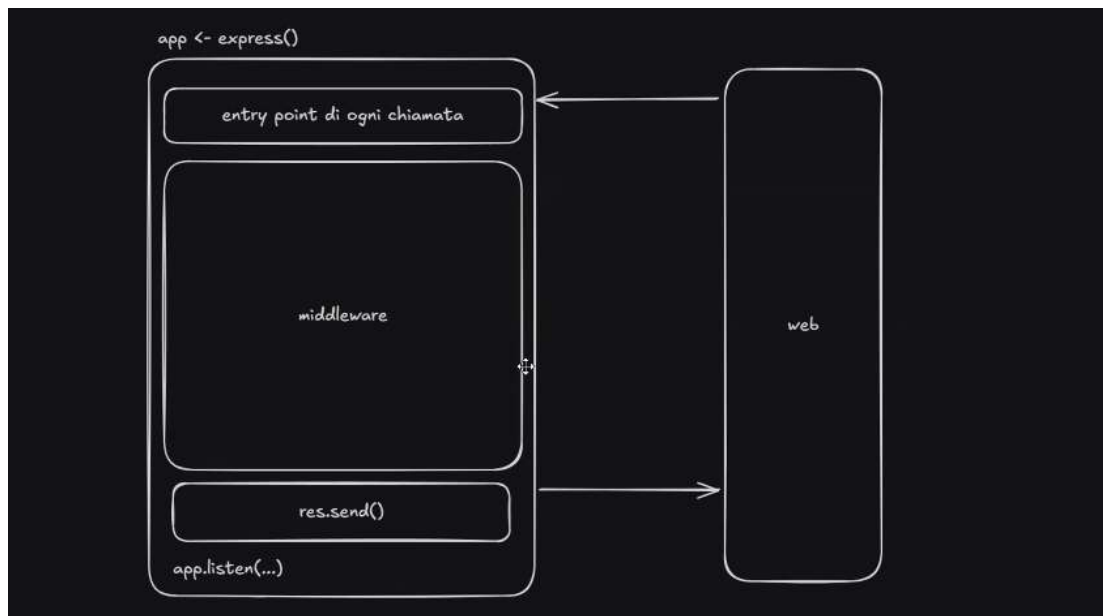
Funzionamento di Express

Variabile app è dove viene allegata / appesa tutta la logica della nostra applicazione - il server è in ascolto su una sola porta e serve un oggetto globale che rappresenti il server intero. tutte le richieste che arrivano hanno lo stesso punto di ingresso (app).

tra la richiesta e la risposta ci sono i middleware (un software tra più software). La funzione di ogni middleware è quella di inviare dati al middleware successivo. L'ultimo è il nostro endpoint.

Middleware vengono introdotti attraverso la funzione app.use() - esempio di uso di middleware:

- autenticazione - se la richiesta risulta legittima, passo la richiesta al middleware successivo
- fornire asset statici



Se è vero che l'entry point è unico, utilizzando le rotte (un po' simile a router di React), si definisce quale endpoint viene colpito dalla nostra richiesta. Normalmente i middleware sono comuni a tutte le rotte ma esistono anche middleware "per rotta".

Qui stiamo andando a costruire le cd. Rest API.

CHIAMATE HTTP

Nel mondo informatico la maggior parte delle comunicazioni avviene tramite il protocollo HTTP, ovvero Hypertext Transfer Protocol. Questo viene utilizzato dal browser per la trasmissione delle pagine web (HTML) e di una serie di altri formati correlati (script, immagini, etc).

Quando è presente la S finale (Secure) la comunicazione avviene con una connessione criptata ottenuta sfruttando crittografia asimmetrica tramite Transport Layer Security (TLS) e relativo certificato installato sul server.

Due fasi:

1. request: Innanzitutto, un client (ad esempio un browser) fa una Richiesta al server ad una rotta specifica, per farsi restituire dei dati o delle intere pagine HTML.
è composta dai seguenti elementi:
 - Method (GET, POST, PUT, PATCH, DELETE, ecc)
 - URL (comprensiva di eventuale query string)
 - Request Headers
 - Request Body (dati inviati al server)
2. response: Successivamente, il server riceve la richiesta e individua a quale rotta è stata fatta. Ogni rotta ha una funzione dedicata e questa funzione deve restituire una Risposta al client, fornendo un testo, dei dati o intere pagine!
E' composta dai seguenti elementi:
 - Status Code (es. 200 - OK, oppure 404 - Page Not Found, etc)

- Response Headers
- Response Body (dati inviati al client/browser)

Una chiamata HTTP è sempre a doppia via.

Chiamate Http hanno sempre lo scopo di gestire delle entità, dette risorse. Il metodo definisce cosa voglio fare con la risorsa in questione (es. dammi la risorsa - GET, crea la risorsa - POST, PATCH e PUT - aggiornamento della risorsa, DELETE - cancella risorsa).

In Express, la request e la response, vengono fornite automaticamente come parametri alla funzione che gestisce una rotta:

```
app.get('/', (req, res) => {
  res.send('Hello World!')
})
```

req: (request) oggetto che rappresenta la richiesta e contiene delle proprietà per accedere alle query string, parametri dinamici, dati ricevuti e molto altro.

res: (response) oggetto che rappresenta la risposta che l'applicazione Express invia a seguito di una richiesta. Permette di specificare i dati da inviare, la loro tipologia o altre operazioni.

RESPONSE

Come ogni server che si rispetti dobbiamo essere in grado di rispondere in diversi modi e di inviare dati o pagine html.

Tramite l'oggetto res che ogni funzione di ogni rotta riceve, possiamo invocare vari metodi:

```
res.send() - invia un contenuto testuale/ html
res.json() - invia i dati sotto forma di json
res.redirect() - esegue un redirect verso un'altra pagina
res.download() - invia un file e ne effettua il download tramite il browser
res.sendFile() - invia un file che verrà aperto direttamente nel browser
```

Ogni metodo richiederà degli argomenti specifici, quindi è essenziale consultare la documentazione di ogni metodo per capire come usarlo.

Esempi:

```
// risponde con Content-Type text/html
res.send(<h1>Hello world</h1>);
```

```
// risponde con Content-Type: application/json
res.json({ message: "Hello world" });
```

Attenzione: quando impostiamo il type('json') o usiamo la scorciatoia res.json(), il metodo send() converte in JSON array e oggetti, ma non altri tipi di dato!

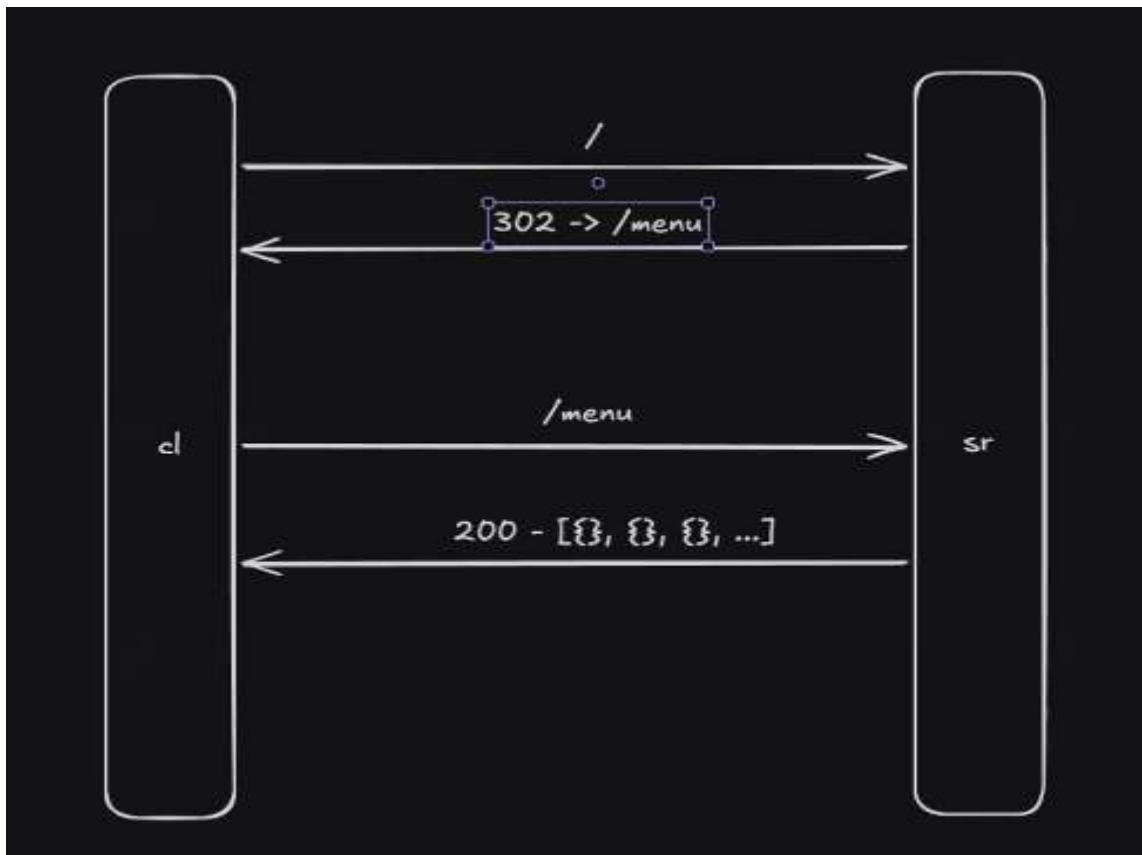
```
// risponde con un redirect alla pagina /altra_pagina  
res.redirect("/altra_pagina")
```

Status code di default:

res.send e res.json - default lo status code è 200

res.redirect - default 3xx

Nota: in caso di redirect il redirect non avviene dentro il server! sono due richieste e due risposte.



ASSET STATICI

Un server non si limita soltanto a restituire e manipolare testi e dati.

Spesso dobbiamo poter leggere interi file statici (immagini, file css, js o altro)

Express di default non attiva questo servizio, pertanto lo dobbiamo attivare noi.

Tramite la funzione app.use() possiamo chiedere ad Express di "abilitare" alcune funzionalità extra!

A questa funzione passiamo come argomento il middleware nativo di express invocando la funzione `express.static()`.

Alla funzione `static` indichiamo come argomento il nome della cartella che conterrà i nostri file statici.

```
app.use(express.static('public'));
```

Infine nella cartella `public` inseriamo qualsiasi file vogliamo utilizzare.

Questi saranno accessibili scrivendo direttamente il dominio del nostro sito seguito dal nome del file.

LEZIONE 01/09/2026

ROUTING

3 strati del full-stack:

- front-end
- backend
- database

App nativamente full stack:

1. CORS: back-end e front end in due posti diversi che si parlano attraverso degli indirizzi pubblici (CORS → cross origin request). backend si comporta come un fornitore di risorse esterno.
2. reverse proxy: frontend fa tutte le richieste sulla stessa origin. Il front-end contatta un backend attraverso percorsi relativi, non fa riferimento a una macchina esterna. C'è poi un proxy (un intermediario) che inoltra le richieste al backend vero e proprio. In questo modo per il client esiste solo un backend (BFF - backend for frontend), ma poi avviene un inoltro "dietro le quinte".

ROUTING

Routing -> mappatura delle varie URI e endpoint (instradamento verso la callback corretta / endpoint corretto)

Il termine Routing o instradamento è il processo di selezione del percorso che una Richiesta HTTP deve prendere.

In altre parole è l'operazione che permette di determinare come l'applicazione deve rispondere a seguito di una Request del client ad un determinato URI (o path) fatta con uno specifico metodo HTTP (GET, POST, ecc...).

Ricordiamo che la porta è unica, ma all'interno della chiamata http del client arriva al server una richiesta di accedere a un determinato endpoint, a cui possiamo arrivare appunto attraverso il routing e il path.

Come tutti i framework, anche Express ha un sistema di routing integrato che facilita questa operazione.

Tramite Express possiamo definire tutti gli indirizzi (rotte) di cui abbiamo bisogno per la nostra applicazione. Per ogni rotta possiamo specificare 3 elementi:

1. metodo HTTP che dovrà essere usato per accedere alla nostra rotta
2. path → URL della rotta
3. funzione che definisce la risposta quando viene chiamata la rotta

```
app.get('/', (req, res) => {  
  res.send("ciao")  
})
```

Remind: req e res vengono forniti automaticamente come parametri alla funzione che gestisce una rotta

req: (request) oggetto che rappresenta la richiesta e contiene delle proprietà per accedere alle query string, parametri dinamici, dati ricevuti e molto altro.

in Express req è un "oggettone" che contiene tante cose al suo interno - il parametro req contiene al suo interno anche le rotte (a cui posso accedere → req.path).

"Oggettone" = oggetto JS su cui sono state messe a bordo sia informazioni utili sia delle vere e proprie API.

res: (response) come già visto, res è l'oggetto che rappresenta la risposta che l'applicazione Express invia a seguito di una richiesta. Permette di specificare i dati da inviare, la loro tipologia o altre operazioni.

Abbiamo già visto la creazione di path semplicemente attraverso app.get e la definizione del tipo e payload della res all'interno del codice della callback

In realtà, questo si potrebbe fare anche senza utilizzare questa semplificazione che ci mette a disposizione Express (non ottimale, ma utile esercizio per comprendere meglio i parametri req e res) → il codice con i due if sotto è equivalente al codice commentato

```

import express from 'express';
import { menu } from './menu.js';

const app = express();
const port = 3000;

app.use(express.static('public'));

app.use((req, res) => {
  if (req.path === '/' && req.method === 'GET') {
    res.redirect('/client');
    return;
  }

  if (req.path === '/menu' && req.method === 'GET') {
    res.json(menu);
    return;
  }
});

// app.get('/', (req, res) => {
//   res.redirect('/client');
// });

// app.get('/menu', (req, res) => {
//   res.json(menu);
// });

app.listen(port, () => {
  console.log(`Server listening on port ${port}`);
});

```

Rotte dinamiche

Oltre a definire il path di una rotta, possiamo prevedere anche dei parametri dinamici. Queste sono porzioni dell'URI che vengono riservate ed utilizzate per recuperare dei valori non statici, comunemente passati dal browser, come l'id di una risorsa → parametri dinamici sono dei segnaposto.

Per definire un parametro dinamico, scegliamo una porzione di URI e ne assegniamo un nome. Precediamo poi il nome con il simbolo : (due punti)

`app.get('/products/:id', (req, res) => { // logica della funzione per gestione rotta })`

All'interno della funzione, possiamo accedere a questo parametro proprio tramite l'oggetto `req`, più precisamente con **`req.params.nome_parametro`** → nell'esempio **`req.params.id`**

Nota: i parametri sono sia posizionali sia nominali - ogni barra del URL si comporta in modo simile a una virgola nei parametri di una funzione.

Quando ho due o più parametri dinamici, questi non possono avere lo stesso nome (nominali), nonostante ciò quando scrivo l'URL e passo l'argomento al posto del parametro dinamico, mi ricava quel parametro dalla posizione dell'argomento (posizionali).

Parametri sono identificati tramite posizione ma resi leggibili all'endpoint tramite il loro nome.

CRUD → Create, Read (fetch), Update, Delete (o kill).

Questi 4 verbi sono le azioni che posso fare sui dati del mio DB. Purtroppo non corrispondono 1 a 1 ai metodi HTTP, ma ogni metodo ci permette di fare una di queste azioni.

per gli amici e goliardicamente anche detti FUCK (fetch, update, create, kill) 😊

REST → Representational State Transfer

REST introduce il concetto di risorse: all'interno di ogni applicazione, abbiamo alcune entità centrali che giocano un ruolo fondamentale nel nostro progetto, che vengono definite **risorse**.

Nell'architettura REST le risorse sono in fin dei conti fonti di informazioni a cui si può accedere tramite un identificatore globale (un URI).

Per utilizzare le risorse, client e server comunicano attraverso una interfaccia standard (HTTP) e si scambiano rappresentazioni di queste risorse.

Di fatto, tramite l'architettura rest, possiamo usare un sistema di comunicazione per rappresentare dei cambiamenti di stato delle risorse: abbiamo le risorse, ognuna con un identificatore, comune a client e server (usato da entrambi per identificarle), e questi si scambiano rappresentazioni di quella risorsa tramite chiamate HTTP.

REST "sta sopra" l'HTTP, ed è per di più una rappresentazione concettuale, ma che guida la creazione di server / web API.

Funzionamento di architettura Rest prevede:

1. Un nome da assegnare alle rotte per ciascuna operazione
2. una struttura degli URL ben definita e univoca per ogni risorsa (più precisamente la parte finale, detta endpoint)
3. l'utilizzo dei metodi HTTP per azioni specifiche sulle risorse

METODI HTTP

GET recupero dati

POST creazione nuovi dati

PUT / PATCH per l'aggiornamento dei dati esistenti → PUT replacement in blocco / PATCH aggiornamento solo delle informazioni cambiate, il resto non cambia

DELETE cancellazione dati

importante: la logica deve essere implementata dentro la funzione di callback, non c'è niente che mi impedisce per esempio di implementare una funzione che cancella un elemento con metodo POST, è puramente convenzionale.

Combinando i Verbi HTTP e gli endpoint, otteniamo queste combinazioni univoche, con le quali creeremo tutte le rotte necessarie alle nostre risorse.

Operazione sulla risorsa	Nome Rotta	Verbo HTTP	Endpoint
Read: Visualizzare tutti gli elementi	Index	GET	pizzas/
Read: Visualizzare un elemento	Show	GET	pizzas/:id
Create: Creare un nuovo elemento	Store	POST	pizzas/
Update: Modificare interamente un elemento	Update	PUT	pizzas/:id
Update: modificare parzialmente un elemento	Modify	PATCH	pizzas/:id
Delete: Eliminare un elemento	Destroy	DELETE	pizzas/:id

*Notiamo che se abbiamo **endpoint uguali**, abbiamo **verbi HTTP differenti**.*

*Notiamo che il **nome della risorsa nell'endpoint è sempre al plurale***

*Notiamo inoltre che **quando bisogna recuperare un singolo elemento**, abbiamo sempre un **parametro dinamico nell'endpoint**!*

Nota: post non ha id nell'endpoint perchè deve essere passato nel payload.

Payload e metodi:

- read deve avere un payload in risposta (res) ma non in chiamata (req)
- store nel nostro caso avrà un payload in chiamata ma non ce l'ha in risposta, a parte un eventuale id in risposta. Alcune API ci restituiscono un payload in risposta a store più grande, includendo per esempio anche un timestamp (created on...) e l'intero oggetto creato con tutti gli attributi aggiuntivi che vengono "appesi" alla risorsa creata.
- Put e patch hanno un payload in chiamata ma non in risposta.
- Delete tendenzialmente nè in chiamata nè in risposta. A volte però ha un payload in risposta che manda indietro l'oggetto cancellato (stesso payload della GET, azione distinta).

Approfondimento - Concetto di idempotenza: se chiamo più volte lo stesso metodo sullo stesso endpoint cambia il risultato oppure no (se non cambia: si dice che il metodo è **idempotente**). GET, PUT e DELETE sono idempotenti. PATCH può essere idempotente oppure no: se per esempio si include un contatore: una richiesta patch potrebbe incrementare il contatore (mentre PUT no, perchè sostituisce l'intera risorsa). POST no, se creo 10 oggetti uguali avrò 10 oggetti uguali. Delete: qualche server potrebbe prevedere una implementazione di delete non idempotente, rispondendo con un error 404 not found in caso si cerchi di cancellare una risorsa già cancellata / non esistente.

Quindi, se vogliamo creare ad esempio le rotte per le nostre pizze, avremo tutte le rotte col nome della risorsa al plurale (/pizzas) e le rotte finali saranno:

[GET] /pizzas (index)
 [GET] /pizzas/:id (show)
 [POST] /pizzas (store)
 [PUT] /pizzas/:id (update)
 [PATCH] /pizzas/:id (modify)
 [DELETE] /pizzas/:id (destroy)

Per ogni risorsa avrò quindi 6 endpoint. Nell'esempio sotto il codice è abbastanza compatto perchè è un'app molto semplice, ma immaginando una app reale diventerebbe pesante e

avrebbe molte linee di codice → Vedi esempio sotto in app.delete dei passaggi intermedi che ci aspettiamo in un'app reale.

```
import express from 'express';
import { menu } from './menu.js';

const app = express();
const port = 3000;

app.get('/pizzas', (req, res) => {
  res.send('Lista delle pizze');
});

app.get('/pizzas/:id', (req, res) => {
  res.send('Dettaglio della pizza con id: ' + req.params.id);
});

app.post('/pizzas', (req, res) => {
  res.status(201).send('Creazione della pizza');
});

app.put('/pizzas/:id', (req, res) => {
  res.status(204).send('Aggiornamento integrale della pizza con id: ' + req.params.id);
});

app.patch('/pizzas/:id', (req, res) => {
  res.send('Aggiornamento parziale della pizza con id: ' + req.params.id);
});

app.delete('/pizzas/:id', (req, res) => {
  // permessi
  // chiamata a db
  // risposta con esito
  res.send('Eliminazione della pizza con id: ' + req.params.id);
});

app.listen(port, () => {
  console.log(`Server listening on port ${port}`);
});
```

ROUTER EXPRESS

Express mi mette a disposizione una utility(router) per poter gestire questi metodi in maniera più ordinata:

Guardando il nostro file, ci rendiamo subito conto che tutte le rotte delle pizze, iniziano con /pizzas

Se avessimo altre risorse, ad esempio le bibite, avremmo poi tante rotte che iniziano con /drinks

Tramite la classe express.Router possiamo creare un file dedicato ad un gruppo di rotte.

In questo file, andremo ad indicare tutte le rotte che riguardano una determinata entità, lasciando più asciutto app.js e organizzando meglio il nostro codice:

Da quel momento in poi, sappiamo che tutte le rotte delle pizze sono nel loro bel file dedicato!

Come fare?

Creiamo una cartella routers all'interno della quale aggiungeremo un file per ogni entità di cui vogliamo raggruppare le rotte.

All'interno di ogni file dentro la cartella routers (vedi sotto):

1. importiamo Express
2. creiamo una variabile router il cui valore sarà un'istanza di `express.Router()`
3. invece di `app`, utilizziamo `router` per definire le rotte
4. Rimuoviamo il nome della risorsa dagli URI (`/pizzas`)
5. esportiamo l'istanza di router

1. `const express = require('express')`
2. **`const router = express.Router();`**

// index

3. **`router.get('/', function(req, res) {`** // 4. notare path senza `/pizzas`
`res.send('Lista delle pizze');`
`});`

// show

3. **`router.get('/:id', function(req, res) {`** // 4. notare path senza `/pizzas`
`res.send('Dettagli della pizza ' + req.params.id);`
`});`

// store

3. **`router.post('/', function(req, res) {`** // 4. notare path senza `/pizzas`
`res.send('Creazione nuova pizza');`
`});`

5. **`module.exports = router;`** // nota: questo è export di common js ma possiamo usare `module`

Dopo, dobbiamo comunicare all'app principale l'esistenza di queste rotte:

1. Importiamo il router appena creato
2. Tramite la funzione `use` indichiamo a `express` che esistono nuove rotte, specificando:
 - Quale deve essere il prefisso (`"/pizzas"`)
 - Quale router utilizzare

```
const express = require("express");
const app = express();
const port = 3000;
const pizzasRouter = require('./routers/pizzas.js');
```

```
app.use(express.static('public'));
```

```
app.get("/", (req, res) => {  
  res.send(`Hello world`);  
});
```

app.use("/pizzas", pizzasRouter)

```
app.listen(port, () => {  
  console.log(`App attiva su http://localhost:${port}`);  
});
```

Nota: Router di fatto è un middleware che creo su un segmento di URL.

La funzione use si aspetta due argomenti:

- path base, comune a tutte le rotte. Questo verrà aggiunto come prefisso ad ogni rotta creata tramite il router
- istanza del router (variabile che abbiamo creato con `express.Router()`)

Approfondimento: controindicazione dell'approccio Routing express e soluzione alternativa

controindicazione dell'approccio sopra è che se devo per esempio ricercare un endpoint specifico dentro al mio codice perché per esempio risulta un bug non lo troverò → immaginiamoci una app molto complessa e mi dicono che c'è un problema su `drinks/:id`. cercando "`drinks/:id`" su Visual Studio non mi uscirà nulla perché gli endpoint sono "rotti" / separati dato che il segmento `drinks` è comune a più endpoint.

1. Modifico il mio [pizzas.js](#) / `drinks.js` dentro la cartella router tornando a usare `app`. invece di `router`.
2. I path torneranno ad essere completi.
3. per convenzione e chiarezza, rinomino la cartella router in "resources".
4. per poter chiamare `app` nel mio file principale ([index.js](#) / [app.js](#) / [main.js](#)) senza fare import ed export "rischiosi", faccio diventare `app` il parametro di una funzione che contiene tutti i miei endpoint.
5. rendo esportabile la funzione.

```

export const registerPizzas = app => {
  app.get('/pizzas/', (req, res) => {
    res.send('Lista delle pizze');
  });

  app.get('/pizzas/:id', (req, res) => {
    res.send('Dettaglio della pizza con id: ' + req.params.id);
  });

  app.post('/pizzas/', (req, res) => {
    res.send('Creazione della pizza');
  });

  app.put('/pizzas/:id', (req, res) => {
    res.send('Aggiornamento integrale della pizza con id: ' + req.params.id);
  });

  app.patch('/pizzas/:id', (req, res) => {
    res.send('Aggiornamento parziale della pizza con id: ' + req.params.id);
  });

  app.delete('/pizzas/:id', (req, res) => {
    res.send('Eliminazione della pizza con id: ' + req.params.id);
  });
};

```

6. nel mio file principale, importo la funzione → nell'esempio sopra la funzione registerPizzas.
7. chiamo la funzione, passando app come argomento.

```

$ server.js > ...
1  import express from 'express';
2  import { registerPizzas } from './resources/pizzas.js';
3  import { registerDrinks } from './resources/drinks.js';
4
5  3 references
6  export const app = express();
7  const port = 3000;
8
9  registerPizzas(app);
10 registerDrinks(app);
11
12 // app.use('/pizzas', pizzasRouter);
13 // app.use('/drinks', drinksRouter);
14
15 app.listen(port, () => {
16   console.log(`Server listening on port ${port}`);
17 });

```

8. una ulteriore overkill che può essere utile in alcuni casi é separare ulteriormente la logica delle funzioni per renderli delle tabelle facilmente ispezionabili (vedi sotto

esempio con [drinks.js](#)). Questo si ottiene scrivendo la logica delle funzioni in un altro file, rendendola esportabile e importandola poi qui:

```
JS drinks.js 2 JS server.js X
resources > JS drinks.js > ...
2 references
1 export const registerDrinks = app => {
2   app.get('/drinks', getAllDrinks);
3   app.get('/drinks/:id', getDrinkById);
4   app.post('/drinks/', createDrink);
5   app.put('/drinks/:id', replaceDrink);
6   app.patch('/drinks/:id', patchDrink);
7   app.delete('/drinks/:id', deleteDrink);
8 };
9
```

LEZIONE 03/09/2026

CRUD

Parte 1 -> Endpoint senza payload nella richiesta: INDEX, SHOW, DESTROY

CRUD INDEX

get per restituire un elenco / array

Esempio

```
import express from 'express'
import menu from './menu.js'
```

```
app.get('/', (req, res) => {
  res.json(menu)
})
```

criteri di filtraggio -> e se volessi filtrare i risultati (tipo ottenere solo le pizze che abbiano la mozzarella?)

3 posti che il client può popolare per dare delle informazioni al server:

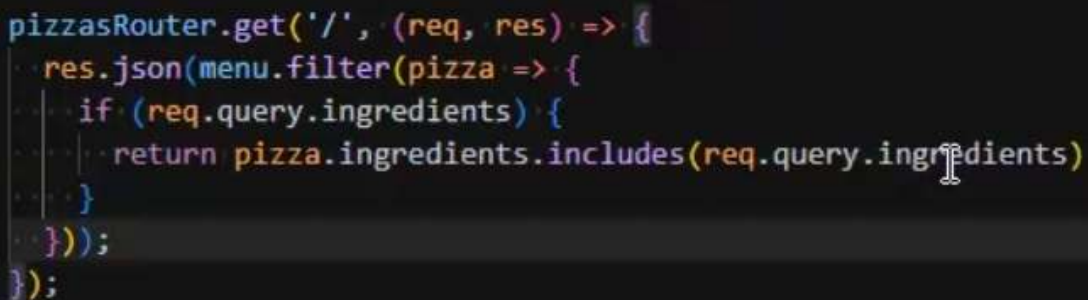
- query string
- parametri dinamici - segmenti dell'URL non statici che stiamo già usando
- Request body - vero e proprio payload della richiesta - ma per index e show non abbiamo un payload della richiesta

per esclusione, criteri di filtraggio stanno nella query string → Express mette a disposizione dentro il parametro req la funzione query → req.query

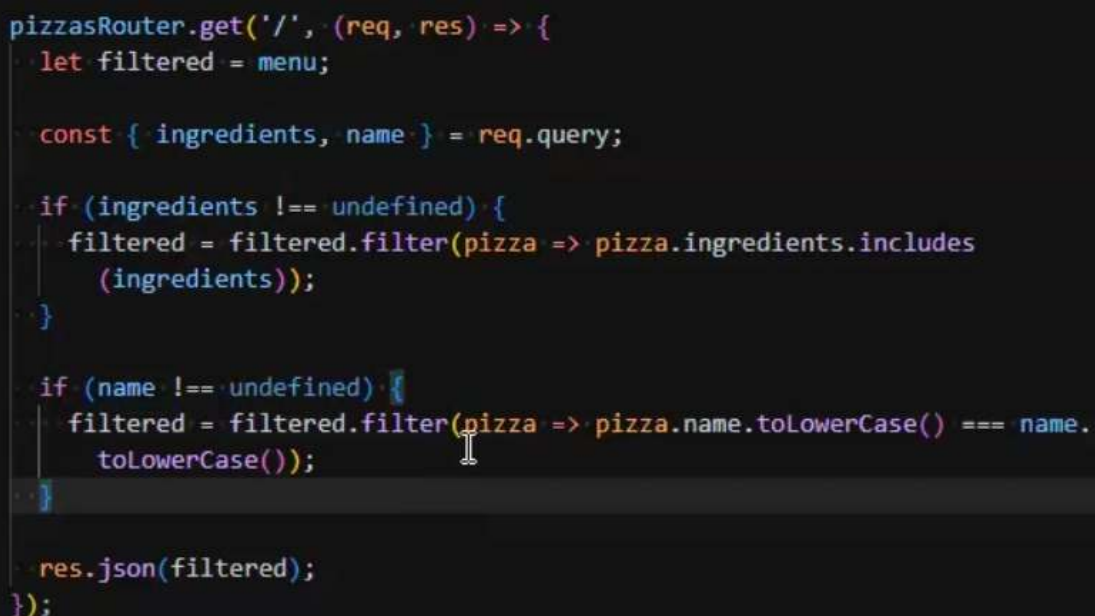
per implementarlo come filtro posso usare metodo filter e poi implementare la logica:

```
// index
router.get('/', function (req, res) {
  //Inizialmente, il menu filtrato corrisponde a quello originale
  let filteredMenu = menu;

  // Se la richiesta contiene un filtro, allora filtriemo il menu
  if (req.query.ingredient) {
    filteredMenu = menu.filter(
      pizza => pizza.ingredients.includes(req.query.ingredient)
    );
  }
  // restituiamo la variabile filteredMenu // potrebbe essere stata filtrata o contenere il menu originale
  res.json(filteredMenu);
});
```



```
pizzasRouter.get('/', (req, res) => {
  res.json(menu.filter(pizza => {
    if (req.query.ingredients) {
      return pizza.ingredients.includes(req.query.ingredients)
    }
  }));
});
```



```
pizzasRouter.get('/', (req, res) => {
  let filtered = menu;

  const { ingredients, name } = req.query;

  if (ingredients !== undefined) {
    filtered = filtered.filter(pizza => pizza.ingredients.includes(
      ingredients));
  }

  if (name !== undefined) {
    filtered = filtered.filter(pizza => pizza.name.toLowerCase() === name.
      toLowerCase());
  }

  res.json(filtered);
});
```

Nota: se inserissi delle query con qualcosa che non esiste, la risposta sarebbe un array vuoto - lo status è 200 OK. Formalmente restituisce sempre un elenco (un array).

CRUD SHOW

get per restituire un solo elemento

```
pizzasRouter.get('/:id', (req, res) => {  
  const id = Number(req.params.id);  
  const pizza = menu.find(pizza => pizza.id === id);  
  
  res.json(pizza);  
});
```

Qui è necessario gestire lo status e farlo diventare 404 se la risorsa non viene trovata

```
pizzasRouter.get('/:id', (req, res) => {  
  const id = Number(req.params.id);  
  const pizza = menu.find(pizza => pizza.id === id);  
  
  if (pizza === undefined) {  
    res.status(404).json({  
      status: 'Not found',  
      message: 'Pizza non trovata'  
    });  
    return;  
  }  
  
  res.json(pizza);  
});
```

CRUD DESTROY

delete un singolo elemento

Per cancellare un elemento, usiamo splice (cancellando l'elemento dall'array). Il metodo delete normalmente come risposta dá un 204 - no content.

Possiamo rispondere direttamente con: `res.sendStatus(204)`.

importante: ogni volta che modifico qualcosa nei miei file js e salvo (o uso `-watch`), riparte il server. Quindi il delete non è permanente.

CONTROLLER

Per evitare che la cartella router diventi troppo pesante e poco leggibile, possiamo delegare la parte logica, che quindi non riguarda strettamente la gestione delle rotte (obiettivo del router) a un file esterno → Cd. Controller

Otteniamo una divisione per ruoli. Ricordiamoci che comunque non abbiamo vincoli rispetto all'organizzazione del nostro codice, express non ha vincoli particolari in tal senso.

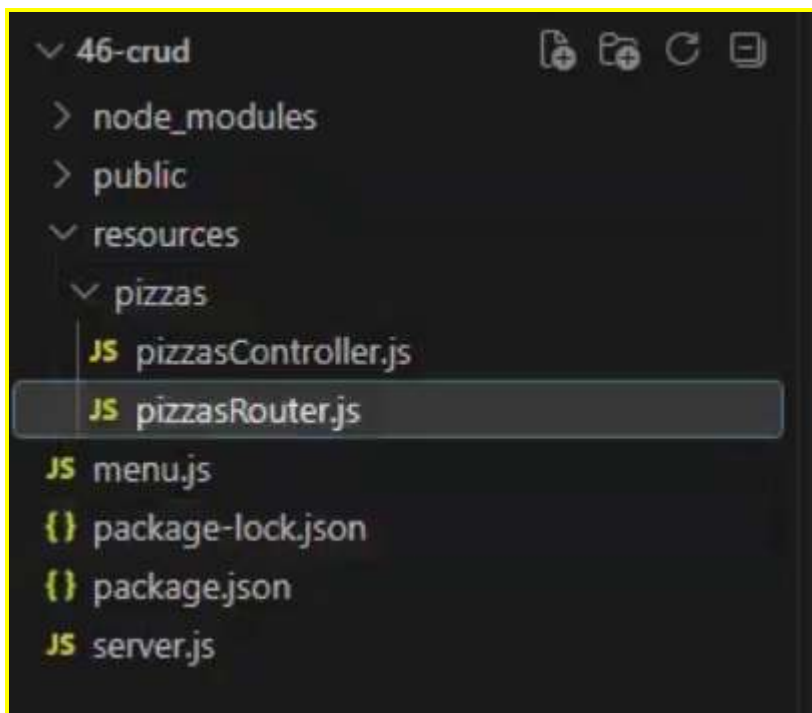
creazione di una cartella controllers e all'interno creiamo le funzioni che ci fanno da callback, con l'obiettivo di avere un codice pulito.

Dopodiché, possiamo rendere esportabili le funzioni e importarle nel file del router.

Approfondimento

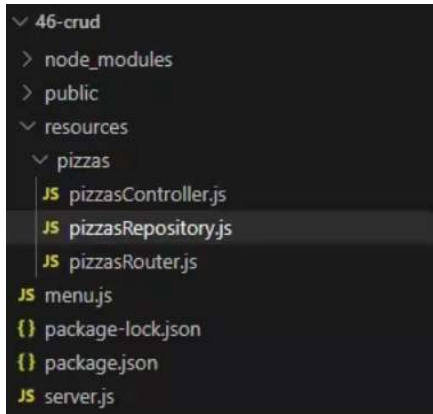
1. Organizzazione alternativa del codice

Un metodo alternativo di organizzazione del codice é creare una cartella per ogni entità dove all'interno salvare il relativo controller e router



2. Repository Pattern

Oltre a voler separare la gestione degli endpoint dalla gestione dei path / routing, potrei voler separare la gestione degli endpoint da un punto di vista server (ovvero tutta la logica della request e response) dalla gestione dell'endpoint da un punto di vista dati (ovvero la logica dal punto di vista dei metodi array che applichiamo / variabili di appoggio, ecc.).



	pizza	drink	allergen
router			
controller			
repository		+	

Router

```
JS server.js x JS pizzasRouter.js x
resources > pizzas > JS pizzasRouter.js > ...
1 import express from 'express';
2 import * as pizzasController from './pizzasController.js';
3
4 8 references
5 export const pizzasRouter = express.Router();
6
7 pizzasRouter.get('/', pizzasController.getAll);
8 pizzasRouter.get('/:id', pizzasController.getById);
9 pizzasRouter.post('/', pizzasController.create);
10 pizzasRouter.put('/:id', pizzasController.update);
11 pizzasRouter.patch('/:id', pizzasController.modify);
12 pizzasRouter.delete('/:id', pizzasController.deleteById);
```

Controller

Controller diventa così:

```
pizzasController.js
sources > pizzas > JS pizzasController.js > [🔍] getAll
1 import { menu } from '../menu.js';
2 import * as pizzasRepository from './pizzasRepository';
3
4 1 reference
5 export const getAll = (req, res) => {
6   const filtered = pizzasRepository.getAll(req.query);
7   res.json(filtered);
8 };
9
```



Marco Be

Repository

separiamo la logica http (req, res) con lo strato dati. I dettagli di come trovo i dati sono qui (es. find o altro) e non nel controller.

```
JS pizzasRouter.js × JS server.js × JS pizzasRepository.js ×
resources > pizzas > JS pizzasRepository.js > ...
1 import { menu } from '../menu.js';
2
3 0 references
4 export const getAll = filters => {
5   let filtered = menu;
6   const { ingredients, name } = filters;
7
8   if (ingredients !== undefined) {
9     filtered = filtered.filter(pizza => pizza.ingredients.includes(
10       ingredients));
11   }
12
13   if (name !== undefined) {
14     filtered = filtered.filter(pizza => pizza.name.toLowerCase().includes(name.
15       toLowerCase()));
16   }
17   return filtered;
18 };
19
```

3. Persistenza Dati con FS (FileSystem) - TO DO

LEZIONE 05/09/2026

CRUD

Parte 2 → endpoint con payload nella req

Ci sono casi in cui occorre inviare al server dei dati più complessi e magari anche criptati, tramite metodi HTTP come POST, PUT o PATCH.

POST / PUT / PATCH → richiedono un payload necessariamente per poter funzionare

Patch a livello puramente formale non dá errore senza payload, essendo una modifica parziale, ma sarebbe una cosiddetta no-op (non succede nulla).

Tramite il body della richiesta HTTP possiamo inviare al server qualsiasi tipo di dato, come json, upload file, testo, ecc.

In questi casi, non possono bastare le query string, per cui si ricorre al body della richiesta.

Come fare?

andremo a leggere il request body, accessibile grazie all'oggetto req che già conosciamo

Express mette a disposizione un middleware che fa il body parsing → “attacca” / “appende” il body alla request. tecnicamente il body parsing è la decodifica del body che viene ricevuto e convertirlo in qualcosa di leggibile, come del testo, un json o un file.

Tramite la funzione app.use() possiamo registrare in express i body-parser che intendiamo utilizzare.

Successivamente, tramite l'oggetto req.body potremo leggere tutti i dati che ci vengono inviati.

```
const express = require("express");
const app = express();
const port = 3000;

// registro il body-parser per "application/json"
app.use(express.json());

app.post("/", (req, res) => {
  // dentro req.body troveremo
  // i dati ricevuti in formato json
  console.log(req.body);

  // ...
});
```

CRUD create

post per creazione di un elemento

Ora che abbiamo recuperato i dati ricevuti dal body della richiesta, torniamo ad occuparci della rotta Store.

- creiamo un nuovo id
- creiamo una nuova pizza coi dati ricevuti.
- La aggiungiamo all'array di pizze.
- impostiamo lo status corretto (201 created)
- La restituiamo la nuova pizza in formato JSON

```
function store(req, res) {  
  // Creiamo un nuovo id incrementando l'ultimo id presente  
  const newId = menu[menu.length - 1].id + 1;  
  
  // Creiamo un nuovo oggetto pizza  
  const newPizza = {  
    id: newId,  
    name: req.body.name,  
    image: req.body.image,  
    ingredients: req.body.ingredients  
  }  
  
  // Aggiungiamo la nuova pizza al menu  
  menu.push(newPizza);  
  
  // controlliamo  
  console.log(menu);  
  
  // Restituiamo lo status corretto e la pizza appena creata  
  res.status(201);  
  res.json(newPizza);  
}
```

Nota: creare correttamente un nuovo id univoco è un compito che spetta al database. Noi stiamo simulando l'operazione in maniera approssimativa a fini meramente didattici

CRUD UPDATE

aggiornamento di un elemento (totale / tutti gli attributi)

La rotta Update è quella deputata alla modifica di un elemento esistente nelle nostre risorse. Per poter identificare quale elemento modificare, utilizzeremo come di consueto un parametro dinamico. Inoltre, anche in questo caso, abbiamo bisogno di passare informazioni al server tramite il body

La prima parte di logica è identica a quanto visto con Show e Destroy:

- recuperiamo il parametro id della pizza
- lo convertiamo in numero.

- utilizziamo find per recuperare la pizza
- controlliamo che esista una pizza con l'id fornito
- Fatto ciò, aggiorniamo la pizza con i dati ricevuti nel body della richiesta.
- Restituiamo la pizza aggiornata in formato JSON.

```
function update(req, res) {
  // recuperiamo l'id dall' URL e trasformiamolo in numero
  const id = parseInt(req.params.id)

  // cerchiamo il pizza tramite id
  const pizza = menu.find(pizza => pizza.id === id);

  // Piccolo controllo
  if (!pizza) {
    res.status(404);

    return res.json({
      error: "Not Found",
      message: "Pizza non trovata"
    })
  }

  // Aggiorniamo la pizza
  pizza.name = req.body.name;
  pizza.image = req.body.image;
  pizza.ingredients = req.body.ingredients;

  // Controlliamo il menu
  console.log(menu)

  // Restituiamo la pizza appena aggiornata
  res.json(pizza);
}
```

Nota PUT → PUT prevede normalmente una modifica completa, per cui è opportuno inserire un controllo per verificare che tutti gli attributi dell'oggetto che voglio modificare siano stati passati nella req.body

Per es:

```
const {name, image, ingredients} = req.body
const OK = name && image && ingredients
```

```
if (!OK) {
  res.json({"status": "400: bad request", "message": "il body della richiesta http è assente o incompleto"})
  return
}
```

CRUD MODIFY

aggiornamento di un elemento (parziale / solo gli attributi che vengono passati nel body della request)

L'operazione di Update può essere effettuata anche col verbo PATCH, e la rotta assume informalmente il nome di modify

PUT: è usato per eseguire un aggiornamento completo della risorsa. In altre parole, quando fai una richiesta PUT, devi fornire tutti i campi necessari per rappresentare la risorsa

PATCH: è utilizzato per aggiornamenti parziali. Con PATCH, puoi inviare solo i campi che vuoi modificare, lasciando invariati gli altri campi della risorsa.

Per fare ciò metterò un controllo con operatore ternario oppure con operatore ?? per fare una fallback sul valore precedente, nel caso in cui un attributo non sia stato passato.

APPROFONDIMENTI - to be completed

```
1  # approfondimenti
2
3  ## concetti
4
5  1. UUID / generazione degli ID
6  1. servizio DB
7  1. servizio logger -> observability / testability
8  1. servizio API esterna -> rick & morty API
9
10 ## architettura
11
12 1. dependency injection per _servizi_
13 1. dipendenze "scoped"
14
15 ## sfide
16
17 1. statistiche sul db
18 1. entità annidate
19 1. chiamata api esterne
20 1. funzioni come API
21
```

Logger - logging si può fare non solo in console → con logging intendiamo l'inserimento di checkpoint dove mostriamo cosa succede all'interno del codice. Concetto di "osservabilità del codice"

API esterna - l'idea è delegare la parte di fetch a un modulo a parte

Dipendenze Scoped - dipendenze che hanno una struttura init - use - release (la inicializzo all'interno di un contesto, la utilizzo e poi la "consumo" / la cancello)

LEZIONE 08/09/2026

MIDDLEWARE

Cosa sono i middleware?

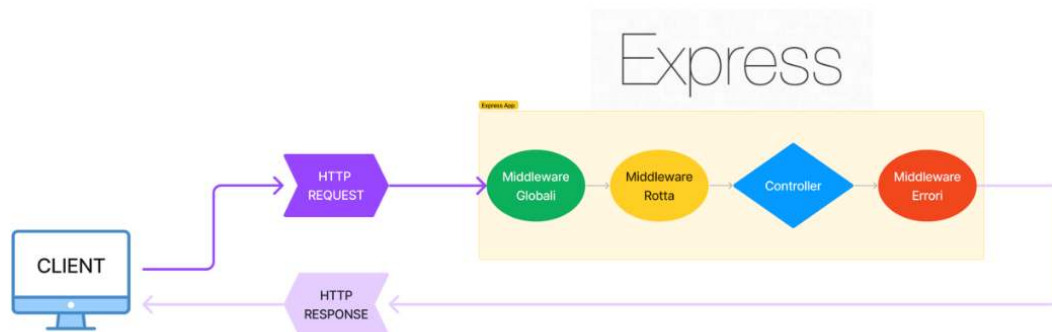
Sono funzioni che vengono eseguite prima che una richiesta HTTP raggiunga il gestore del percorso o prima che un client riceva una risposta, dando al framework la possibilità di eseguire uno script prima o dopo la richiesta del client.

Grazie a questi possiamo intercettare il flusso dell'applicazione. Possiamo ad esempio verificare se un utente è loggato o se ha i permessi per eseguire una determinata azione o accedere ad una pagina.

Anche i middleware di fatto in Express sono call back, come gli endpoint.

Rispondono bene o male alla stessa logica e criteri delle funzioni endpoint.

A seconda del tipo, possono essere eseguiti prima o dopo della call back di una determinata rotta



Middleware possono funzionare come un imbuto / filtro che inoltra o meno allo strato successivo della pipeline --> funnel

Middleware globali --> app.use

Middleware rotta --> router.use

Middleware specifici per endpoint / singolo controller --> esempio è autenticazione

Middleware per la gestione degli errori e dello status (es. 404 o fallback con redirect)

Un middleware può essere usato in un solo punto della pipeline oppure in più parti.

Uso dei middleware

L'utilizzo dei middleware, si svolge in 2 steps:

- 1) Dichiarazione della funzione middleware → Qui definiamo la sua logica
- 2) Registrazione (o assegnazione) del middleware → Questo step serve per far sapere ad Express che un middleware esiste e per quali rotte deve essere utilizzato

I middleware sono delle semplici funzioni (nominate o anonime) che ricevono 3 (o 4, vedasi sotto) argomenti:

- parametri req e res rappresentano rispettivamente la Richiesta e Risposta HTTP
- parametro next rappresenta la prossima funzione che dovrà essere eseguita nel flusso della richiesta

Il numero dei parametri e la forma che prende la funzione e l'utilizzo dei parametri distingue tra middleware standard, middleware errori o endpoint

```
function checkTime(req, res, next) {  
  // se l'ora è dopo le 22:00 e prima delle 7:00  
  // rispondiamo con un messaggio di errore  
  const currHour = new Date().getHours();  
  
  if (currHour >= 22 || currHour < 7) {  
    return res.send("Meglio se vai a dormire!");  
  }  
  
  next();  
}
```

```
module.exports = checkTime
```

Una volta che ho registrato il middleware (in questo caso con la funzione checkTime), dove viene messo all'interno del flusso farà la differenza, perché questo middleware potrà sbarrare la strada e uscire in anticipo a un certo punto del flusso

Utilizzo di next()

La funzione next() si occupa di far proseguire il flusso della richiesta e se non viene invocata, la richiesta rimane in stallo.

Qualora volessimo bloccare una richiesta intenzionalmente è importante invocare il metodo res.send() o simili, per inviare una response che informa il client del motivo per cui l'abbiamo bloccata.

per poter usare un middleware, è necessario registrarlo.

A livello globale

I middleware possono essere registrati a livello globale tramite la funzione use().

Es.

```
const express = require('express')  
const app = express()  
const checkTime = require('./middlewares/checkTime')
```

```
// registrato per tutte le rotte dell'app  
app.use(checkTime)
```

Middleware per path e sotto-percorso

Oltre a specificare i middleware per singola rotta possiamo anche specificarli in base al percorso, indipendentemente dalla rotta.

Alla funzione use() come primo argomento passiamo il path che vogliamo monitorare.

La rotta e tutte le rotte figlie di quel path utilizzano il middleware indicato indipendentemente dal metodo HTTP utilizzato.

Per es:

```
const express = require('express')
const app = express()
```

```
//registrato per tutte le rotte che iniziano con /pizzas
app.use("/pizzas", checkTime);
```

Per singola rotta ed endpoint

Come detto, è possibile anche registrare il middleware (o i middlewares) dentro una funzione endpoint sulla singola rotta.

Per registrarli a livello di singola rotta dovremo specificare il middleware come secondo argomento della funzione che la definisce.

In questo caso il middleware viene eseguito esclusivamente per quella specifica rotta!

```
const express = require('express')
const app = express()
const checkTime = require('./middlewares/checkTime')
```

```
// registrato solo per questa rotta
app.get("/", checkTime, (req, res) => {
  res.send("Hello World!");
});
```

Middleware per router

I middleware possono essere anche essere registrati a livello di router tramite la funzione use().

Ad esempio, assegnandolo al router delle pizze, verrebbe eseguito per ogni rotta che riguarda le pizze, ma non per le altre!

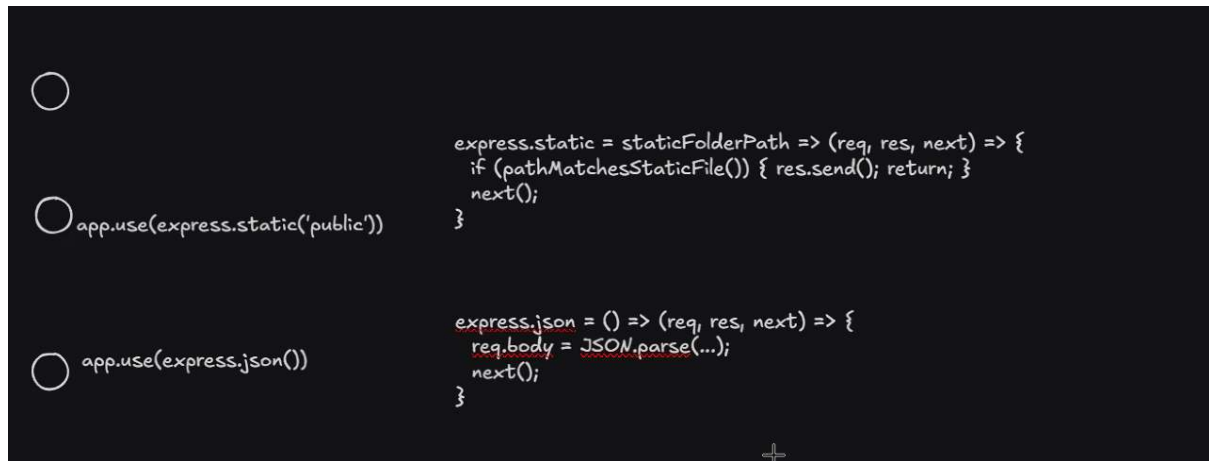
Per es:

```
const express = require('express')
const app = express()
const checkTime = require('./middlewares/checkTime')
const router = express.Router()
```

```
// registrato per le rotte di questo router
router.use(checkTime)
```

nota: body-parser e asset statici, così come il router, come già anticipato, sono middleware

express.static = cortocircuita la chiamata Http --> Fornisce uno dei middleware che non fa arrivare all'endpoint. In realtà express.static è una sorta di fabbrica di middleware, essendo una funzione che genera middleware, ma non un middleware, che ha questa struttura:



express.json = anche express.json è una funzione che genera un middleware - express.json non è un middleware

Middleware Errori

Proviamo a usare i middleware per intercettare i possibili errori dell'applicazione.

Creeremo una funzione come quella degli altri middleware, ma questa avrà 4 argomenti, dove il primo sarà `err` mentre gli altri saranno `req`, `res` e `next`.

```
function errorHandler(err, req, res, next) {  
  res.status(500)  
  res.json({  
    error: err.message,  
  });  
};
```

```
module.exports = errorHandler;
```

Infine registriamo il middleware globalmente e in fondo a tutte le altre rotte:

```
const express = require("express");  
const app = express();  
const errorHandler = require("../middlewares/errorHandler");
```

```
// Tutte le altre rotte...
```

```
// Poi  
app.use(errorHandler);
```

Curiosità per errorHandler → come fa express ha sapere quanti parametri ha una funzione e capire che si tratta di un error handler middleware quando lo passo a app.use?--> tramite function.length che restituisce il numero di parametri --> nota: vengono tenuti conto solo i parametri fissi (non destrutturati).

Possiamo anche creare una funzione che gestisca l'errore in caso venga chiamata una rotta inesistente:

Creiamo un middleware che risponda adeguatamente, con un messaggio d'errore e uno status 404 - Not Found

```
function notFound (req, res, next) {  
  res.status(404)  
  res.json({  
    error: "Not Found",  
    message: "Pagina non trovata"  
  });  
};
```

```
module.exports = notFound;  
const express = require("express");  
const app = express();  
const notFound = require("./middlewares/notFound");
```

```
// Tutte le altre rotte...
```

```
// Poi  
app.use(notFound);
```

Successivamente registriamo il middleware globalmente e in fondo a tutte le altre rotte, anche dopo quella di gestione errori

Notiamo come in questo caso:

- non usiamo err perchè per Express non è un errore chiamare una rotta inesistente
- non usiamo next() perchè non vogliamo che la richiesta prosegua ma che risponda con 404